Exp:N0.1

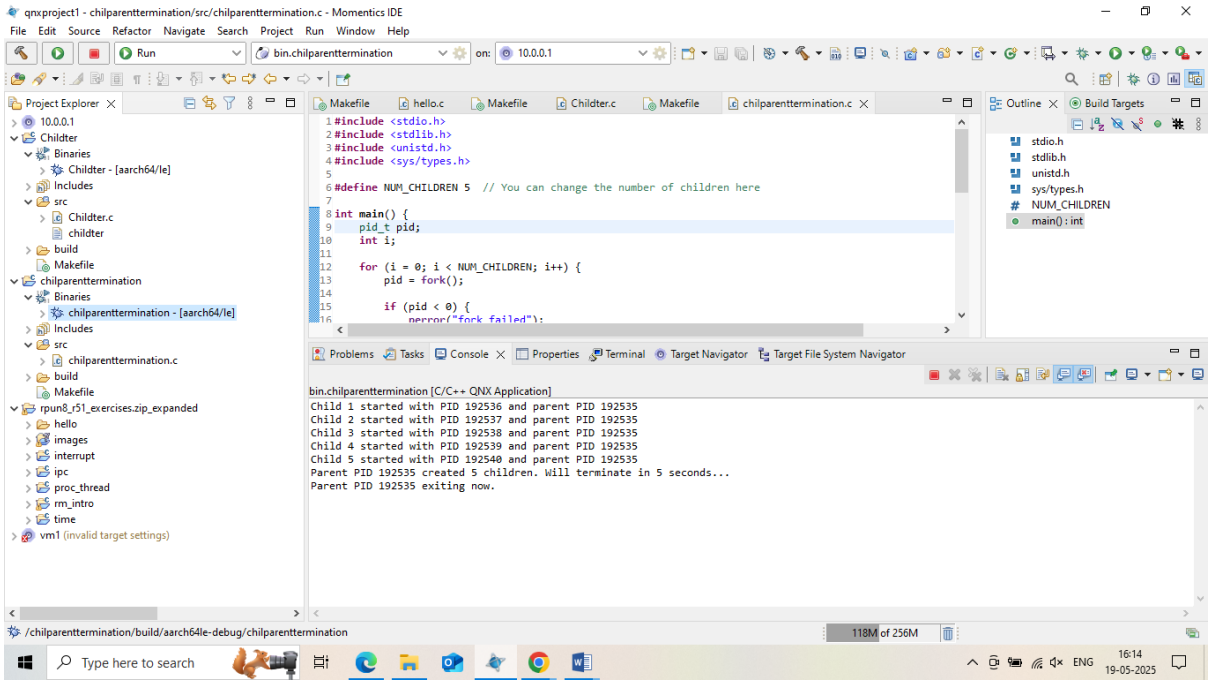
Create and run hello world example on VM Ware virtual machine or Rasppi board.

Exp:N0.2

Write a program that creates multiple child processes using fork() and print different messages in parent and child process. Terminate the parent after 5 seconds and print the pid from the child.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

#define NUM_CHILDREN 5  // You can change the number of children here

int main() {
    pid_t pid;
    int i;

    for (i = 0; i < NUM_CHILDREN; i++) {
        pid = fork();

        if (pid < 0) {
            perror("fork failed");
            exit(1);
        } else if (pid == 0) {
            // Child process
            printf("Child %d started with PID %d and parent PID %d\n", i + 1,
getpid(), getppid());

            // Loop until parent dies
            while (1) {
                sleep(1);
                pid_t ppid = getppid();

                if (ppid == 1) {  // Parent is dead; usually reparented to init
                    printf("Child PID %d: Detected parent termination. My new
parent PID is %d\n", getpid(), ppid);
                    break;
                }
```

```
            }

            exit(0);  // Child exits
        }
        // Parent continues to create next child
    }

    // Parent process
    printf("Parent PID %d created %d children, Will be terminate in 5
seconds...\n", getpid(), NUM_CHILDREN);
    sleep(5);
    printf("Parent PID %d exiting now.\n", getpid());
    exit(0);  // Parent terminates
}
```

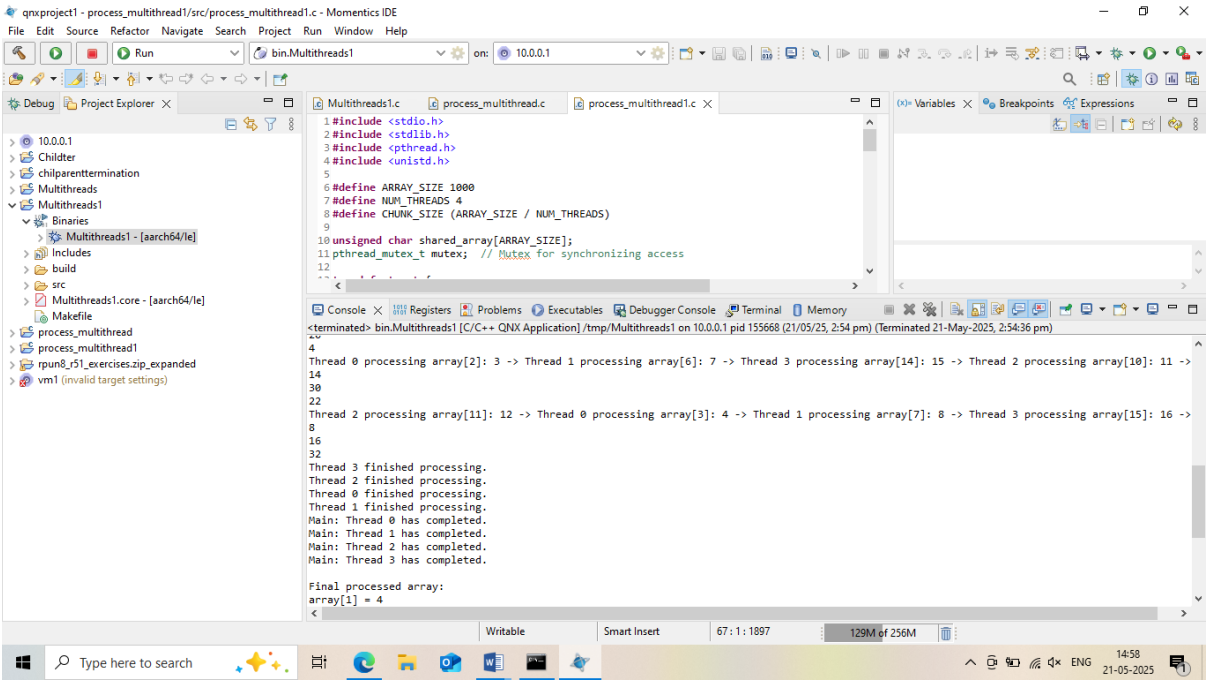


Exp:N0.3

Implement a multi-threaded application using POSIX threads (pthread_create). Each thread should process a different part of an array and the main thread should wait for all threads to complete using pthread_join

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define ARRAY_SIZE 16
#define NUM_THREADS 4

int array[ARRAY_SIZE];
int partial_sums[NUM_THREADS];  // Each thread stores its result here

typedef struct {
    int thread_id;
    int start_idx;
    int end_idx;
} thread_data_t;

void* sum_array_segment(void* arg) {
    thread_data_t* data = (thread_data_t*) arg;
    int sum = 0;

    for (int i = data->start_idx; i < data->end_idx; i++) {
        sum += array[i];
    }

    partial_sums[data->thread_id] = sum;
    printf("Thread %d processed elements [%d, %d) -> partial sum = %d\n",
           data->thread_id, data->start_idx, data->end_idx, sum);

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    thread_data_t thread_data[NUM_THREADS];

    // Initialize the array with sample values
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i + 1;  // 1 to 100
    }

    int chunk_size = ARRAY_SIZE / NUM_THREADS;

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_data[i].thread_id = i;
        thread_data[i].start_idx = i * chunk_size;
        thread_data[i].end_idx = (i == NUM_THREADS - 1) ? ARRAY_SIZE : (i + 1) *
chunk_size;

        if (pthread_create(&threads[i], NULL, sum_array_segment, &thread_data[i])
!= 0) {
            perror("Failed to create thread");
            exit(EXIT_FAILURE);
        }
    }

    // Wait for all threads to finish
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
```

```
    }

    // Combine results
    int total_sum = 0;
    for (int i = 0; i < NUM_THREADS; i++) {
        total_sum += partial_sums[i];
    }

    printf("Total sum of array elements = %d\n", total_sum);

    return 0;
}
```



Exp:N0.4

Write a program to create a process with 4 threads that update the portin of array of size 1000 bytes by updating 250 bytes each. Make the main thread to join on the 4 threads and print the completion. Use mutex to prevent data corruption while each thread is updating the array

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define ARRAY_SIZE 1000
#define NUM_THREADS 4
#define CHUNK_SIZE (ARRAY_SIZE / NUM_THREADS)
```

```c
unsigned char shared_array[ARRAY_SIZE];
pthread_mutex_t mutex;  // Mutex for synchronizing access

typedef struct {
    int thread_id;
    int start_index;
    int end_index;
} ThreadData;

// Thread function
void* update_array(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    printf("Thread %d started: updating indices %d to %d\n", data->thread_id,
data->start_index, data->end_index - 1);

    for (int i = data->start_index; i < data->end_index; i++) {
        pthread_mutex_lock(&mutex);  // Ensure only one thread updates at a time
        shared_array[i] = (unsigned char)(data->thread_id + 1);  // Set byte to
thread ID + 1
        pthread_mutex_unlock(&mutex);
        usleep(100);  // Optional: simulate processing delay
    }

    printf("Thread %d completed.\n", data->thread_id);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    ThreadData thread_data[NUM_THREADS];

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        shared_array[i] = 0;
    }

    // Initialize mutex
    if (pthread_mutex_init(&mutex, NULL) != 0) {
        perror("Mutex initialization failed");
        exit(1);
    }

    // Create 4 threads
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_data[i].thread_id = i;
        thread_data[i].start_index = i * CHUNK_SIZE;
        thread_data[i].end_index = (i + 1) * CHUNK_SIZE;

        if (pthread_create(&threads[i], NULL, update_array, &thread_data[i]) != 0)
{
            perror("Failed to create thread");
            exit(1);
        }
    }

    // Join all threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
        printf("Main: Joined with thread %d\n", i);
```

```
    }

    // Cleanup
    pthread_mutex_destroy(&mutex);

    printf("\nMain: All threads have completed. Final array updated.\n");

    return 0;
}
```

Exp:N0.5

Implementing a thread-safe bounded buffer (also known as a circular queue) that is shared between multiple producer threads and multiple consumer threads. The buffer has a fixed size (N slots). Producers add items to the buffer, and consumers remove items from the buffer.

Code:

```
#include <pthread.h>
#include <unistd.h>
#include <malloc.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <string.h>

struct queueNode
{
	struct queueNode* next_ptr;
	int data;
};

typedef struct queueNode queueNode_t;

int* get_data_and_remove_from_queue();
void write_to_hardware (int* data);
void add_to_queue(int data);
int* get_data_and_remove_from_queue();
int add_element_to_queue(int data);
int print_queue();

int q_n_items; // for illustration purposes, current elements in the queue, this
is accessed in a potentially thread un-safe manner
queueNode_t* dataQueuep; // Shared resource for two threads
int data_ready;                    // is there data in the queue?

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void * hardwareHandler(void *);
void * dataProvider(void *);
```

```
int main(void)
{
	data_ready=0;
	dataQueuep = NULL;

	/* pthread_mutex_init(mutex, NULL);		Can do this instead of
PTHREAD_MUTEX_INITIALIZER
	 * pthread_condvar_init(cond, NULL);	Can do this instead of
PTHREAD_MUTEX_INITIALIZER
	 */

	if ((pthread_create(NULL, NULL, hardwareHandler, NULL)) != EOK)
	{
		fprintf(stderr, "hardwareHandler pthread_create failed.\n");
		exit(EXIT_FAILURE);
	}

	if ((pthread_create(NULL, NULL, dataProvider, NULL)) != EOK)
	{
		fprintf(stderr, "dataProvider pthread_create failed.\n");
		exit(EXIT_FAILURE);
	}

	sleep(20);
	return 0;
}

void * hardwareHandler(void *i)
{
	int status;
	int *data=NULL;

	while (1)
	{
		status = pthread_mutex_lock (&mutex);		// get exclusive
access
		if (status!=EOK)
		{
			fprintf(stderr, "pthread_mutex_lock failed: %s\n",
strerror(status));
			exit(EXIT_FAILURE);
		}
		while (!data_ready)
		{
			status = pthread_cond_wait (&cond, &mutex);  // we wait here
			if (status!=EOK)
			{
				fprintf(stderr, "pthread_cond_wait failed: %s\n",
strerror(status));
				exit(EXIT_FAILURE);
			}
		}


		/* get and decouple data from the queue */
		while ((data = get_data_and_remove_from_queue ()) != NULL)
		{
			status = pthread_mutex_unlock (&mutex);
			if (status!=EOK)
```

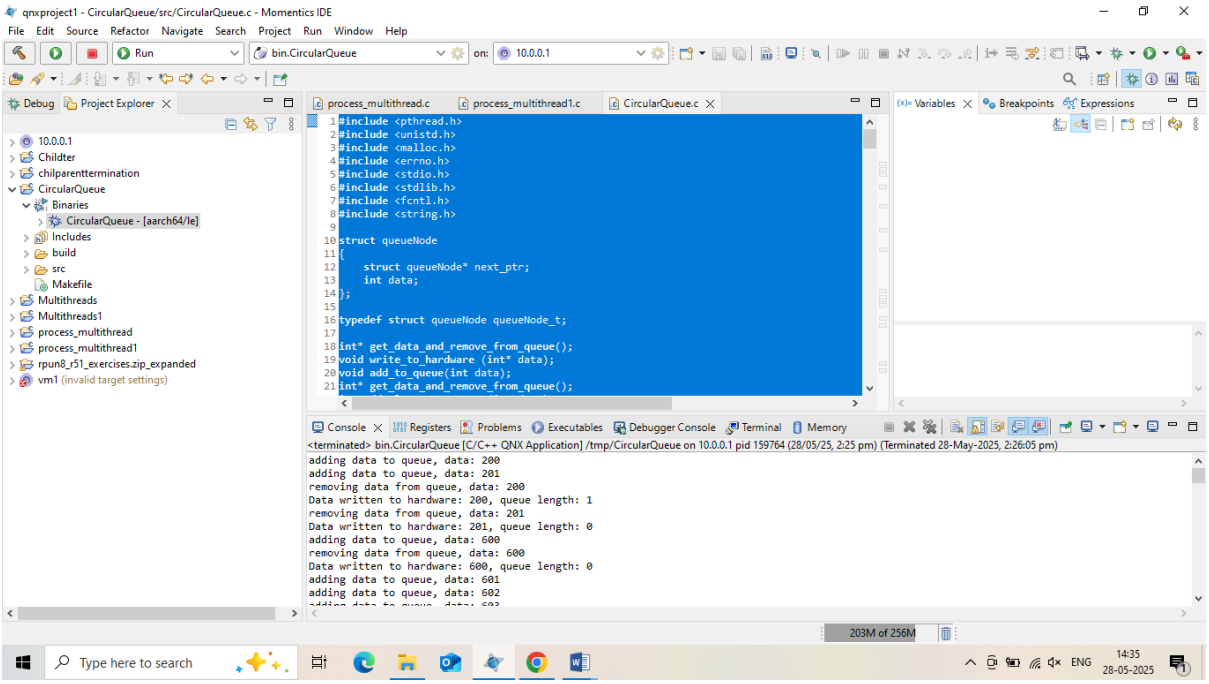
```
                        {
                                fprintf(stderr, "pthread_mutex_lock failed: %s\n",
strerror(status));
                                exit(EXIT_FAILURE);
                        }
                        write_to_hardware (data); // generate output
                        free (data);              // we don't need it after this
                        status = pthread_mutex_lock (&mutex);
                        if (status!=EOK)
                        {
                                fprintf(stderr, "pthread_mutex_lock failed: %s\n",
strerror(status));
                                exit(EXIT_FAILURE);
                        }
                }
                data_ready = 0;                 // reset flag
                status = pthread_mutex_unlock (&mutex);
                if (status!=EOK)
                {
                        fprintf(stderr, "pthread_mutex_unlock failed: %s\n",
strerror(status));
                        exit(EXIT_FAILURE);
                }
                /* do further work */
        }
        return NULL;
}

void * dataProvider(void *unused)
{
        srand(getpid());
        int r_sleep, n_loops, i;

        while(1)
        {
                n_loops = rand() % 10;
                for ( i = 0; i < n_loops; i++ )
                {
                        add_to_queue( n_loops * 100 + i );
                        sched_yield();  // for demonstration purposes, allow the
reader thread a chance to run
                }

                r_sleep = rand() %15;
                delay( r_sleep*100 + 1);

        }
        return NULL;
}

/*
 * add_to_queue
 *
*/
void add_to_queue(int data)
{
        int status;

        status = pthread_mutex_lock (&mutex);     // get exclusive access
```

```
	if (status!=EOK)
	{
		fprintf(stderr, "pthread_mutex_lock failed: %s\n",
strerror(status));
		exit(EXIT_FAILURE);
	}
	add_element_to_queue(data);
	data_ready = 1;			// set the flag

	status = pthread_mutex_unlock (&mutex);	// release exclusivity
	if (status!=EOK)
	{
		fprintf(stderr, "pthread_unmutex_lock failed: %s\n",
strerror(status));
		exit(EXIT_FAILURE);
	}
	status = pthread_cond_signal (&cond);	// notify a waiter
	if (status!=EOK)
	{
		fprintf(stderr, "pthread_cond_signal failed: %s\n",
strerror(status));
		exit(EXIT_FAILURE);
	}
	return;
}

/*
 * add_element_to_queue
*/
int add_element_to_queue(int data)
{
	queueNode_t* newp;
	queueNode_t* endofqueuep=dataQueuep;

	printf("adding data to queue, data: %d\n", data);

	newp = malloc(sizeof(queueNode_t));
	if (newp == NULL)
	{
		errno = ENOMEM;
		exit(EXIT_FAILURE);
	}

	newp->next_ptr = NULL;
	newp->data = data;

	if (dataQueuep == NULL)
	{
		dataQueuep = newp;
	}
	else
	{
		while(endofqueuep->next_ptr != NULL)
		{
			endofqueuep = endofqueuep->next_ptr;
		}
		endofqueuep->next_ptr = newp;
	}
	q_n_items++;
```

```
	return EOK;
}

/*
 * get_data_and_remove_from_queue
*/
int* get_data_and_remove_from_queue()
{
	int* theData=NULL;

	if( dataQueuep )
	{
		// at least one element in the queue

		theData = malloc(sizeof(int));   // Make a place to put the data
		if (theData == NULL)
		{
			errno = ENOMEM;
			exit (EXIT_FAILURE);
		}

		// not empty queue, therefore we can pull the top element off.

		queueNode_t* curp=dataQueuep;    // Get the location of the front of
the queue
		printf("removing data from queue, data: %d\n", curp->data);
		*theData=curp->data;                    // Get the data

		dataQueuep=dataQueuep->next_ptr; // Move the head of the queue down
one spot to remove the element
		free(curp);                                     // Free the queue
element
		q_n_items--;
		//print_queue();
	}

	return theData;
}

void write_to_hardware (int* data)
{
	char buf[255];

	int ret;

	ret = sprintf( buf, "Data written to hardware: %d, queue length: %d\n",
*data, q_n_items ); // unsafe access to q_n_items
	ret = write( STDOUT_FILENO, buf, ret );

	if (ret == -1)
	{
		perror("write ");
		exit(EXIT_FAILURE);
	}
	delay(2);
}

int print_queue()
{
```

```
        queueNode_t* current_ptr = dataQueuep;

        if (dataQueuep == NULL)
        {
                printf("no data in queue\n");
        }
        else
        {
                printf("queue of data (front to back): \n");

                while (current_ptr != NULL)
                {
                        printf("data: %d ", current_ptr->data);
                        current_ptr = current_ptr->next_ptr;
                }
        }
        printf("\n");

        return 0;
}
```

Output:



Exp:N0.6

Create a server process that registers to receive pulses. Write a client that sends pulses to the server using MsgSendPulse, and have the server handle different pulse codes.

Question: Write a client that sends pulses to the server using MsgSendPulse, and have the server handle different pulse codes .

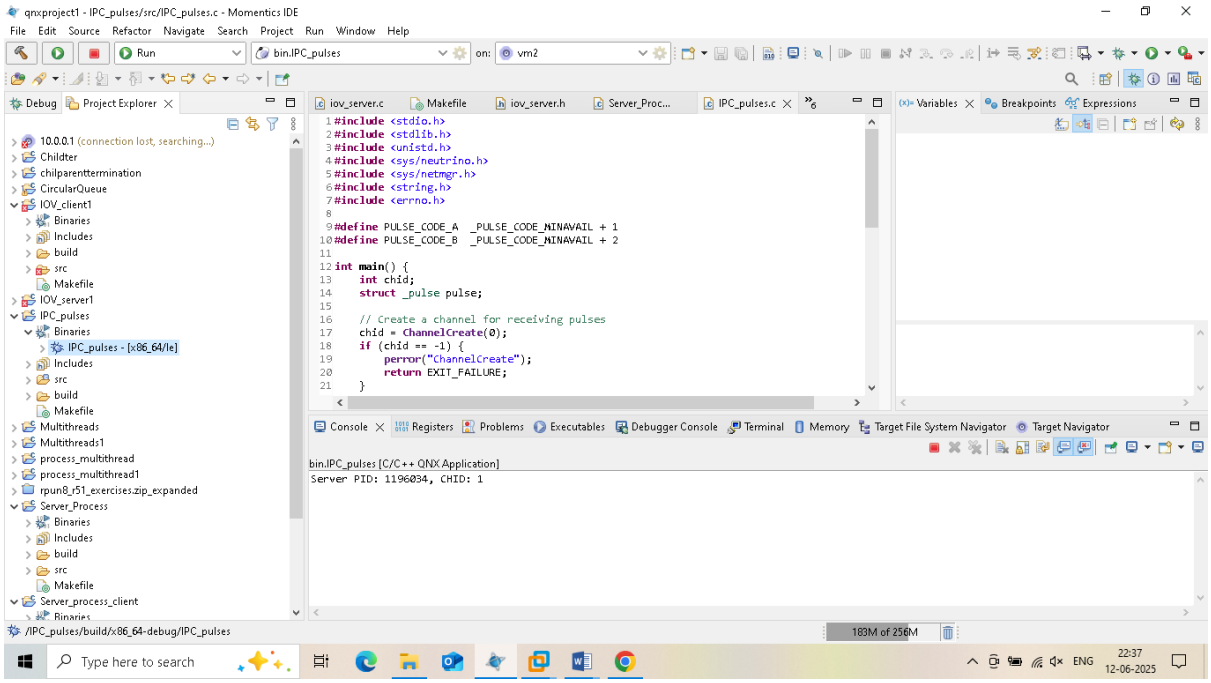
Solution:

First run the server process and it will be waiting pulses,a **server** process that registers to receive pulses and handles them based on different pulse codes,a **client** process that sends pulses using MsgSendPulse().

Server Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/neutrino.h>
#include <sys/netmgr.h>
#include <string.h>
#include <errno.h>

#define PULSE_CODE_A   _PULSE_CODE_MINAVAIL + 1
#define PULSE_CODE_B   _PULSE_CODE_MINAVAIL + 2

int main() {
    int chid;
    struct _pulse pulse;

    // Create a channel for receiving pulses
    chid = ChannelCreate(0);
    if (chid == -1) {
        perror("ChannelCreate");
        return EXIT_FAILURE;
    }

    printf("Server PID: %d, CHID: %d\n", getpid(), chid);

    while (1) {
        // Wait for a pulse (or message)
        int rcvid = MsgReceive(chid, &pulse, sizeof(pulse), NULL);
        if (rcvid == -1) {
            perror("MsgReceive");
            continue;
        }

        // Pulse messages have rcvid == 0
        if (rcvid == 0) {
            switch (pulse.code) {
                case PULSE_CODE_A:
                    printf("Received PULSE_CODE_A with value: %d\n",
pulse.value.sival_int);
                    break;
                case PULSE_CODE_B:
                    printf("Received PULSE_CODE_B with value: %d\n",
pulse.value.sival_int);
                    break;
                default:
                    printf("Received unknown pulse code: %d\n",
pulse.code);
                    break;
            }
```

```
        } else {
            // Handle normal messages if needed
            MsgError(rcvid, ENOSYS);
        }
    }

    return EXIT_SUCCESS;
}
```



Server output:

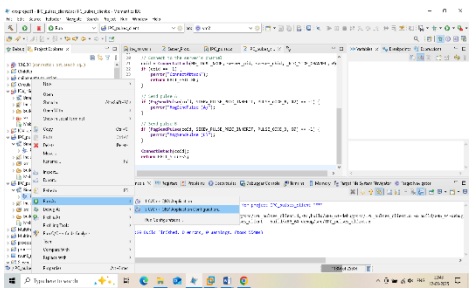
```
Server PID: 1196034, CHID: 1
```

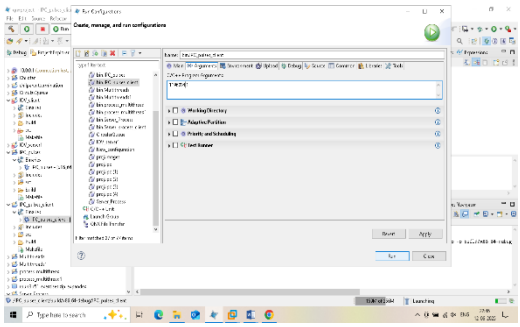
Send PID and CHID details to Client through passing Arguments

Client Program:

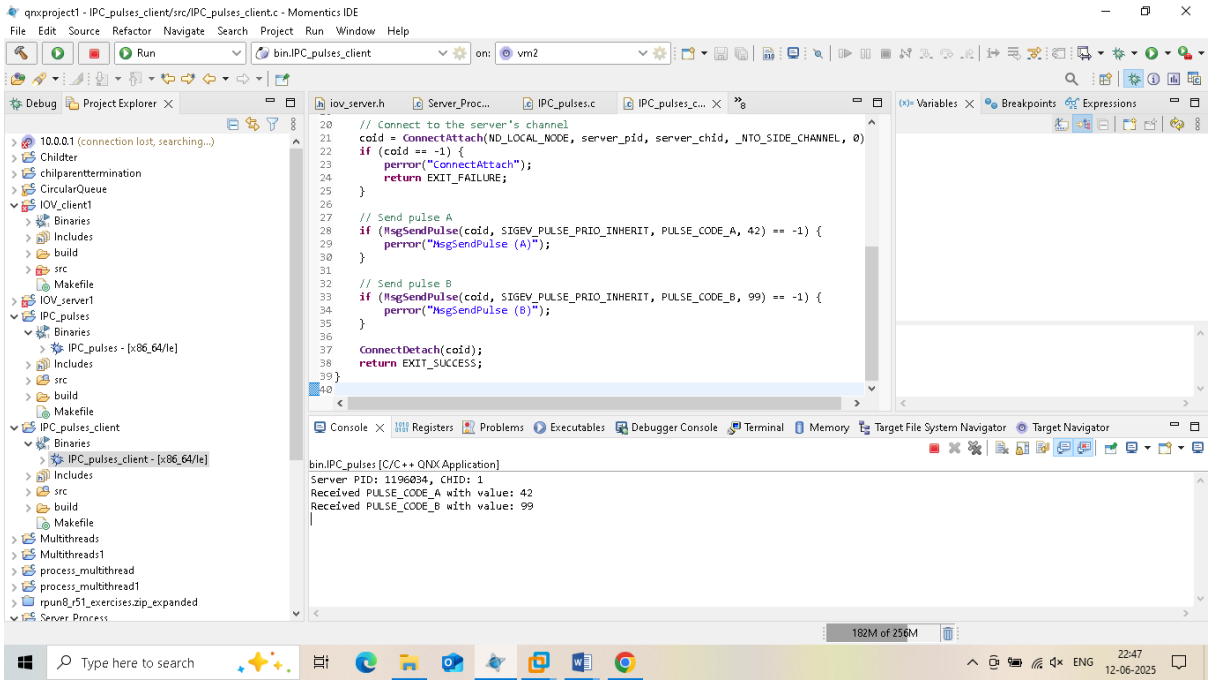
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/neutrino.h>
#include <errno.h>

#define PULSE_CODE_A  _PULSE_CODE_MINAVAIL + 1
#define PULSE_CODE_B  _PULSE_CODE_MINAVAIL + 2

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <server_pid> <server_chid>\n", argv[0]);
        return EXIT_FAILURE;
```

```
    }

    pid_t server_pid = atoi(argv[1]);
    int server_chid = atoi(argv[2]);
    int coid;

    // Connect to the server's channel
    coid = ConnectAttach(ND_LOCAL_NODE, server_pid, server_chid,
_NTO_SIDE_CHANNEL, 0);
    if (coid == -1) {
        perror("ConnectAttach");
        return EXIT_FAILURE;
    }

    // Send pulse A
    if (MsgSendPulse(coid, SIGEV_PULSE_PRIO_INHERIT, PULSE_CODE_A, 42) == -
1) {
        perror("MsgSendPulse (A)");
    }

    // Send pulse B
    if (MsgSendPulse(coid, SIGEV_PULSE_PRIO_INHERIT, PULSE_CODE_B, 99) == -
1) {
        perror("MsgSendPulse (B)");
    }

    ConnectDetach(coid);
    return EXIT_SUCCESS;
}
```





Output:



Exp:N0.7

Develop two processes that communicate using QNX message passing (MsgSend, MsgReceive, MsgReply). The client shall send 3 types of messages and the client shall perform different operation based on the message type and reply to the client.

Solution:

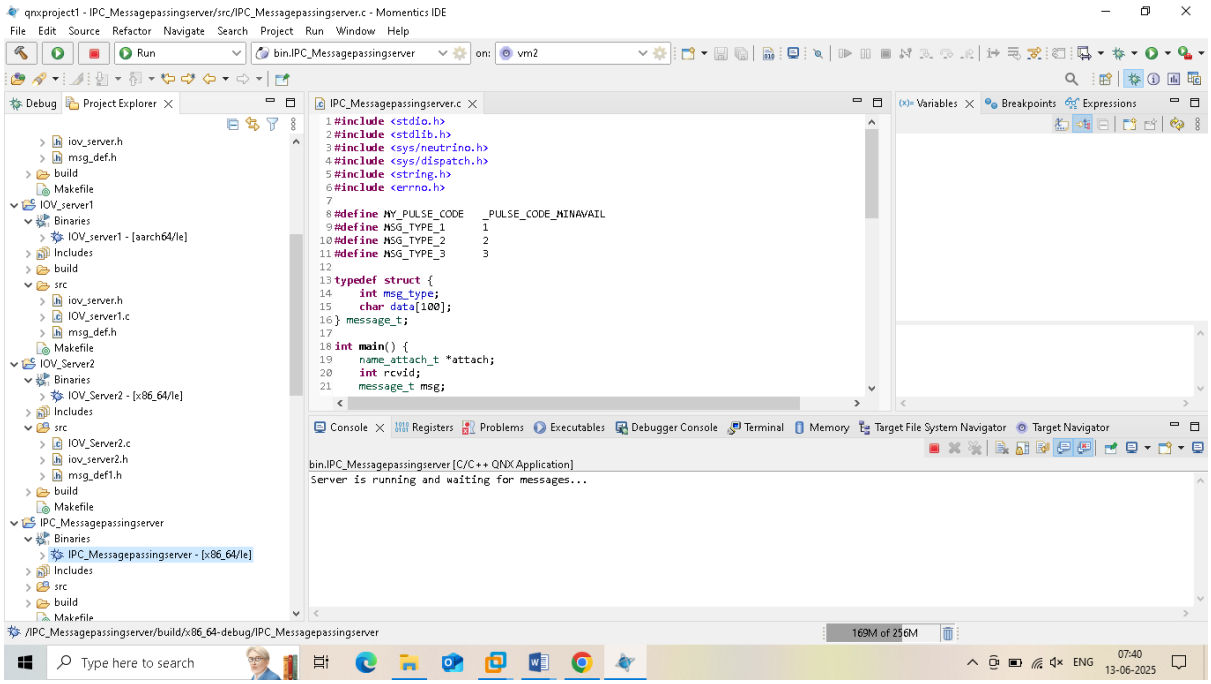
QNX message passing (MsgSend, MsgReceive, MsgReply) is going to establish the IPC communication between server process and client process, which can be run in **Momentics IDE**. This setup consists of two programs: a **server** and a **client**. The server handles three message types from the client and replies accordingly.

Server.C

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/neutrino.h>
#include <sys/dispatch.h>
#include <string.h>
#include <errno.h>

#define MY_PULSE_CODE   _PULSE_CODE_MINAVAIL
#define MSG_TYPE_1      1
```

```c
#define MSG_TYPE_2      2
#define MSG_TYPE_3      3

typedef struct {
    int msg_type;
    char data[100];
} message_t;

int main() {
    name_attach_t *attach;
    int rcvid;
    message_t msg;

    attach = name_attach(NULL, "my_server", 0);
    if (attach == NULL) {
        perror("name_attach");
        exit(EXIT_FAILURE);
    }

    printf("Server is running and waiting for messages...\n");

    while (1) {
        rcvid = MsgReceive(attach->chid, &msg, sizeof(msg), NULL);
        if (rcvid == -1) {
            perror("MsgReceive");
            continue;
        }

        printf("Server received message: Type %d, Data: %s\n",
msg.msg_type, msg.data);

        // Perform operations based on message type
        switch (msg.msg_type) {
            case MSG_TYPE_1:
                MsgReply(rcvid, 0, "Received type 1", strlen("Received type
1") + 1);
                break;
            case MSG_TYPE_2:
                MsgReply(rcvid, 0, "Received type 2", strlen("Received type
2") + 1);
                break;
            case MSG_TYPE_3:
                MsgReply(rcvid, 0, "Received type 3", strlen("Received type
3") + 1);
                break;
            default:
                MsgReply(rcvid, 0, "Unknown message type", strlen("Unknown
message type") + 1);
                break;
        }
    }

    name_detach(attach, 0);
    return 0;
}
```

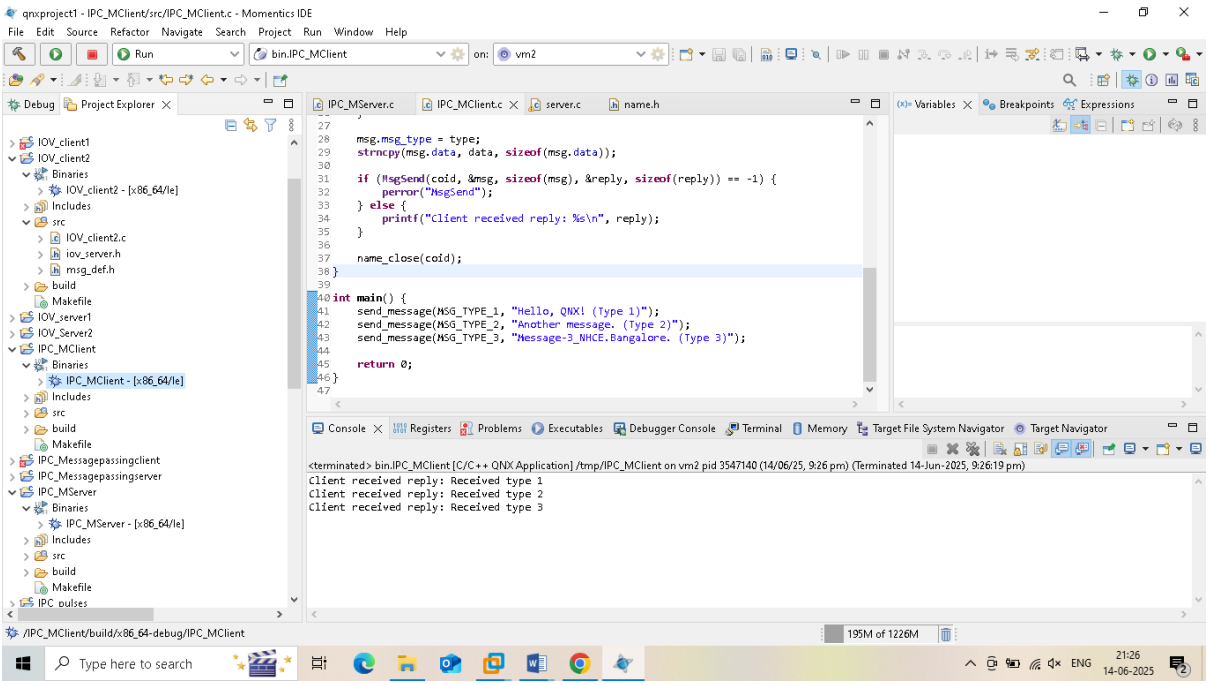
Output: Server is running and waiting for messages…



Client.C

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/neutrino.h>
#include <sys/netmgr.h>
#include <errno.h>

#define MSG_TYPE_1 1
#define MSG_TYPE_2 2
#define MSG_TYPE_3 3

typedef struct {
    int msg_type;
    char data[100];
} message_t;

void send_message(int type, const char *data) {
    int coid;
    message_t msg;
    char reply[100];

    coid = name_open("my_server", 0);
    if (coid == -1) {
        perror("name_open");
        exit(EXIT_FAILURE);
    }

    msg.msg_type = type;
    strncpy(msg.data, data, sizeof(msg.data));

    if (MsgSend(coid, &msg, sizeof(msg), &reply, sizeof(reply)) == -1) {
        perror("MsgSend");
```

```
    } else {
        printf("Client received reply: %s\n", reply);
    }

    name_close(coid);
}

int main() {
    send_message(MSG_TYPE_1, "Hello, QNX! (Type 1)");
    send_message(MSG_TYPE_2, "Another message. (Type 2)");
    send_message(MSG_TYPE_3, "Message-3_NHCE.Bangalore. (Type 3)");

    return 0;
}
```



Save and compile (Ctrl+S, Ctrl+B) for both IPC.Mserver.c and IPM.Mclient.c

Select Binaries file is created after compile.

Select the binaries file right click -> Runas-> C++ Application

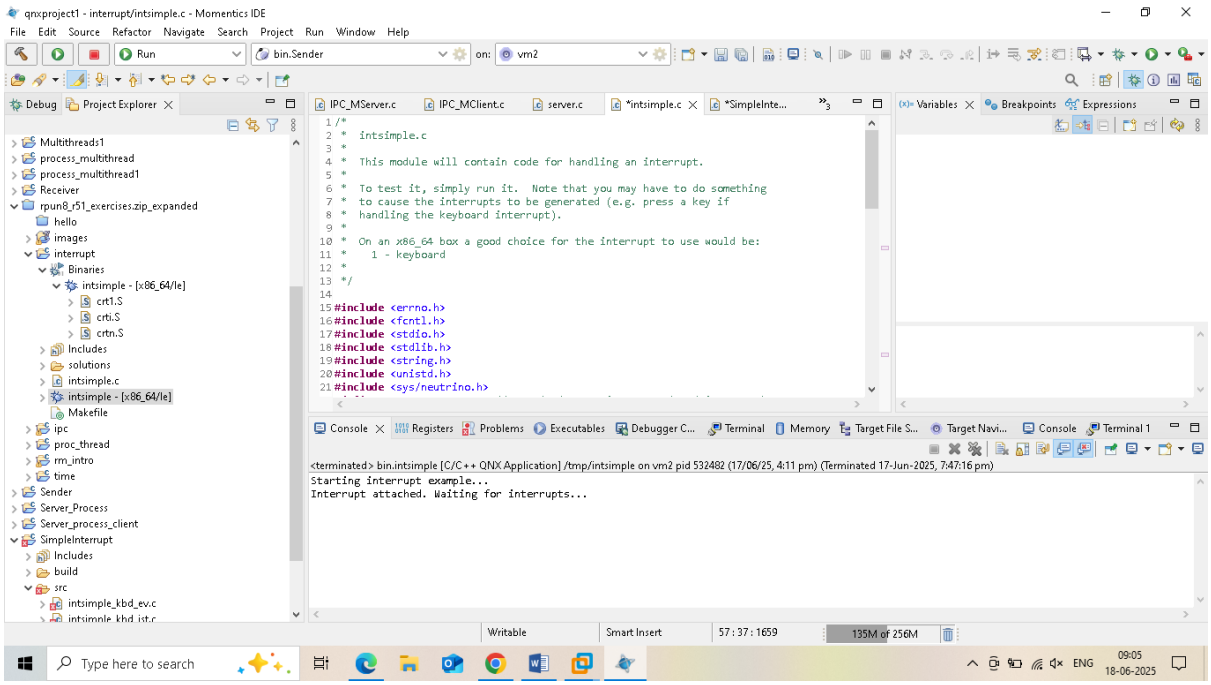
Exp:N0.8

Write a program that uses QNX event notification (sigevent) to notify a process when a timer expires or an interrupt occurs. Demonstrate handling the event in the process. Enable the receiving process to modify the event and reply back to the calling process.

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/neutrino.h>
#define INTERRUPT_LINE 1  // Standard Input interrupt (used in QNX VM)

volatile int done = 0;

// === IST function ===
const struct sigevent* ist_handler(void* arg, int id) {
    // Just return SIGEV_INTR to wake the waiting thread
    return &((struct sigevent*)arg)[0];
}

int main(int argc, char **argv) {
    printf("Starting interrupt example...\n");

    struct sigevent event;
    int interrupt_id;

    // === Initialize the sigevent to unblock InterruptWait ===
    SIGEV_INTR_INIT(&event);

    // === Attach the interrupt line 1 to the handler ===
    interrupt_id = InterruptAttachEvent(
        INTERRUPT_LINE,      // IRQ 1 (usually keyboard in VM)
        &event,              // Event to signal when interrupt occurs
        _NTO_INTR_FLAGS_TRK_MSK); // Allow multiple hits

    if (interrupt_id == -1) {
        perror("InterruptAttachEvent");
        exit(EXIT_FAILURE);
    }

    printf("Interrupt attached. Waiting for interrupts...\n");

    // === Loop to wait for interrupt ===
    while (1) {
        if (InterruptWait(0, NULL) == -1) {
            perror("InterruptWait");
            break;
```

```
        }
        printf("Interrupt received! (IRQ %d)\n", INTERRUPT_LINE);
    }

    // === Detach interrupt (unreachable here, but good practice) ===
    InterruptDetach(interrupt_id);

    return 0;
}
```

Output in QNX Console:
```
Starting interrupt example...
Interrupt attached. Waiting for interrupts...
```

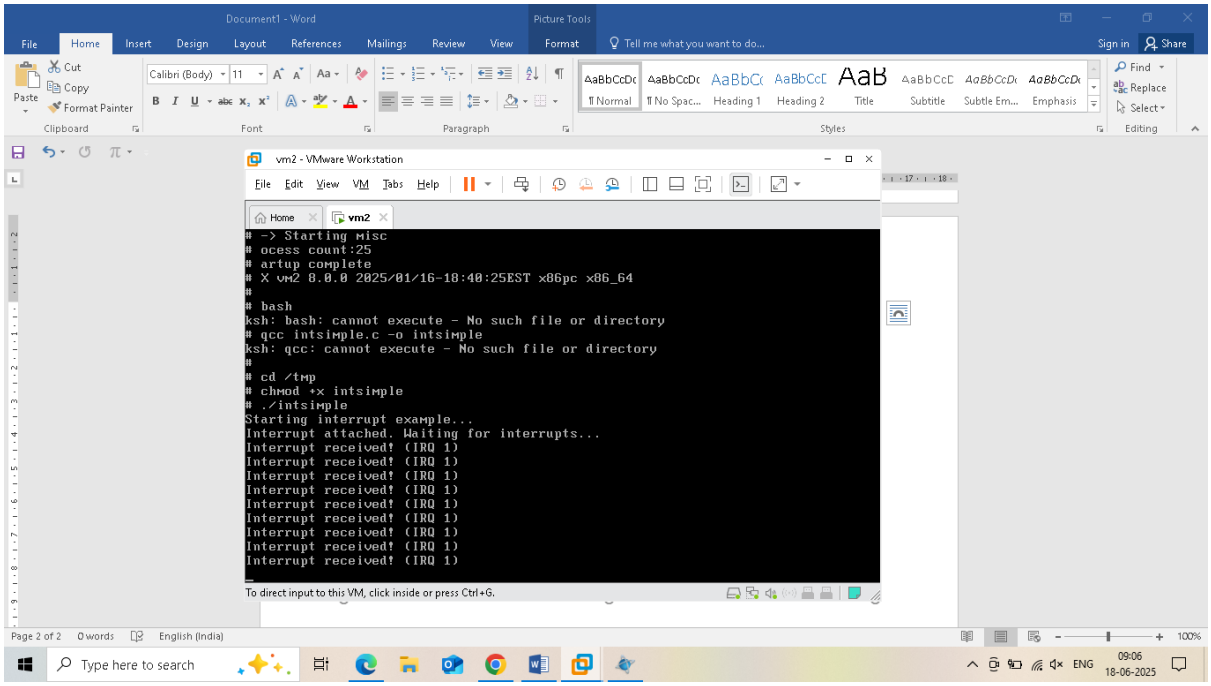


Output: QNX VM console: Starting interrupt example….

Interrupt attached, waiting for interrupts.

After pressing any key on QNX VM console

It will display: Interrupt received (IRQ 1)



Exp:N0.9

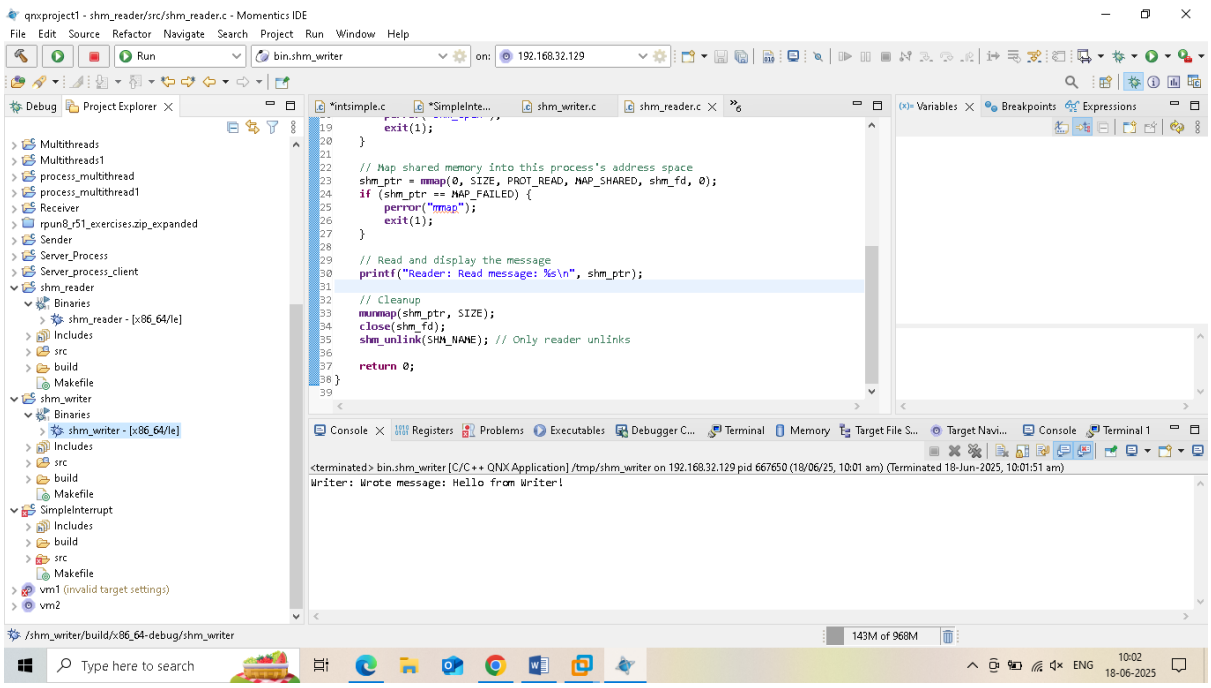
Implement two processes that communicate via shared memory using shm_open and mmap. One process writes data, and the other reads and displays it.

Writer:

```c
// writer.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>      // For shm_open
#include <sys/mman.h>   // For mmap
#include <unistd.h>     // For ftruncate

#define SHM_NAME "/my_shm"
#define SIZE 4096

int main() {
    int shm_fd;
    char *shm_ptr;

    // Create shared memory object
    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        exit(1);
    }

    // Set the size of the shared memory
    if (ftruncate(shm_fd, SIZE) == -1) {
```

```
        perror("ftruncate");
        exit(1);
    }

    // Map the shared memory into this process's address space
    shm_ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED) {
        perror("mmap");
        exit(1);
    }

    // Write data to shared memory
    const char *message = "Hello from Writer!";
    snprintf(shm_ptr, SIZE, "%s", message);

    printf("Writer: Wrote message: %s\n", message);

    // Cleanup: keep shm_fd open so reader can access it
    return 0;
}
```

Output: `Writer: Wrote message: Hello from Writer!`



Reader:

```
// reader.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
```

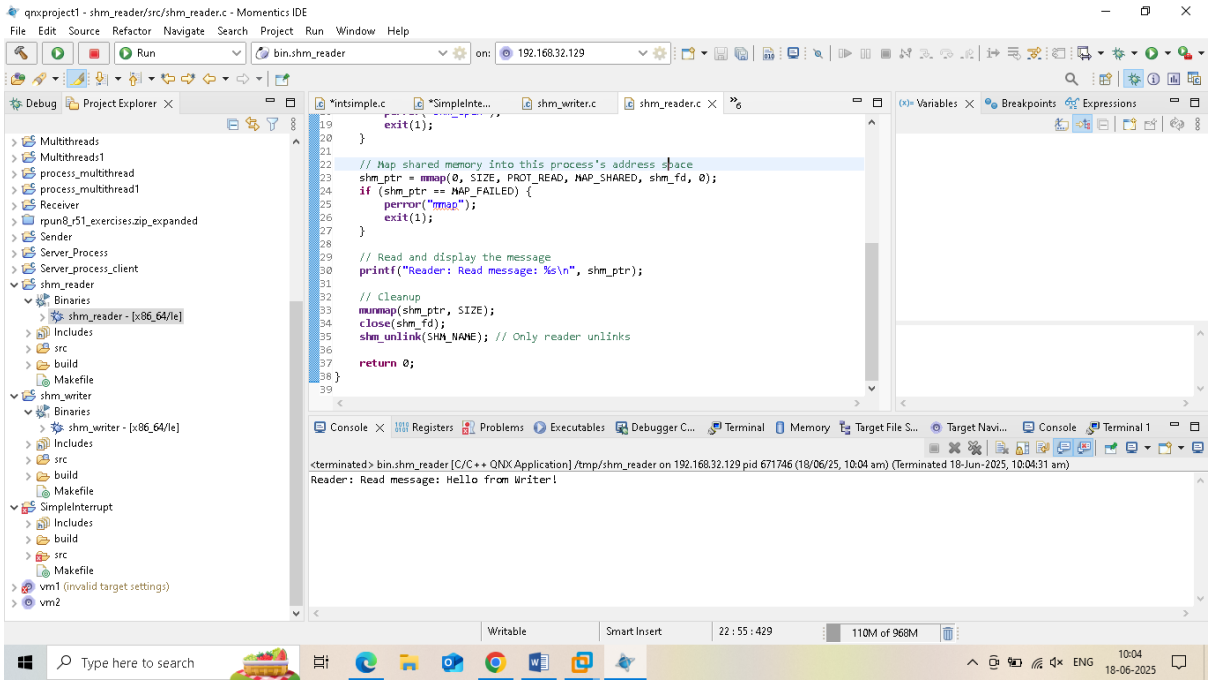
```c
#include <sys/mman.h>
#include <unistd.h>

#define SHM_NAME "/my_shm"
#define SIZE 4096

int main() {
    int shm_fd;
    char *shm_ptr;

    // Open existing shared memory object
    shm_fd = shm_open(SHM_NAME, O_RDONLY, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        exit(1);
    }

    // Map shared memory into this process's address space
    shm_ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED) {
        perror("mmap");
        exit(1);
    }

    // Read and display the message
    printf("Reader: Read message: %s\n", shm_ptr);

    // Cleanup
    munmap(shm_ptr, SIZE);
    close(shm_fd);
    shm_unlink(SHM_NAME); // Only reader unlinks

    return 0;
}
```

Open Target Navigator

Add a new QNX target:

1. Right-click in Target Navigator → `New QNX Target System`
2. Fill details:
    - VM's IP address
    - **Target type**: QNX Neutrino
    - **Username**:
    - **Password**: leave blank (unless set)
3. Click **Finish**

Output: `Reader: Read message: Hello from Writer!`



Exp:N0.10

Create a program that sets up a periodic timer using timer_create and timer_settime. The timer should be used to track the time and kill the process after 10 seconds of execution.

Create a program that sets up a periodic timer using timer_create and timer_settime. The timer should be used to track the time and kill the process after 10 seconds of execution.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <errno.h>
```

```
#include <sys/neutrino.h>
#include <time.h>

#define TIMER_PULSE_CODE (_PULSE_CODE_MINAVAIL + 1)
#define TIMEOUT_SECONDS 10

typedef union {
    struct _pulse pulse;
} message_t;

int main(void) {
    int chid, coid, rcvid;
    struct sigevent event;
    struct itimerspec timer_spec;
    timer_t timerid;
    message_t msg;
    int seconds_elapsed = 0;

    // Step 1: Create a channel
    chid = ChannelCreate(0);
```

```
if (chid == -1) {
    perror("ChannelCreate");
    return EXIT_FAILURE;
}

// Step 2: Connect to the channel
coid = ConnectAttach(0, 0, chid, _NTO_SIDE_CHANNEL, 0);
if (coid == -1) {
    perror("ConnectAttach");
    ChannelDestroy(chid);
    return EXIT_FAILURE;
}

// Step 3: Set up the pulse event
SIGEV_PULSE_INIT(&event, coid, SIGEV_PULSE_PRIO_INHERIT, TIMER_PULSE_CODE, 0);

// Step 4: Create the timer
if (timer_create(CLOCK_MONOTONIC, &event, &timerid) == -1) {
    perror("timer_create");
    ConnectDetach(coid);
```

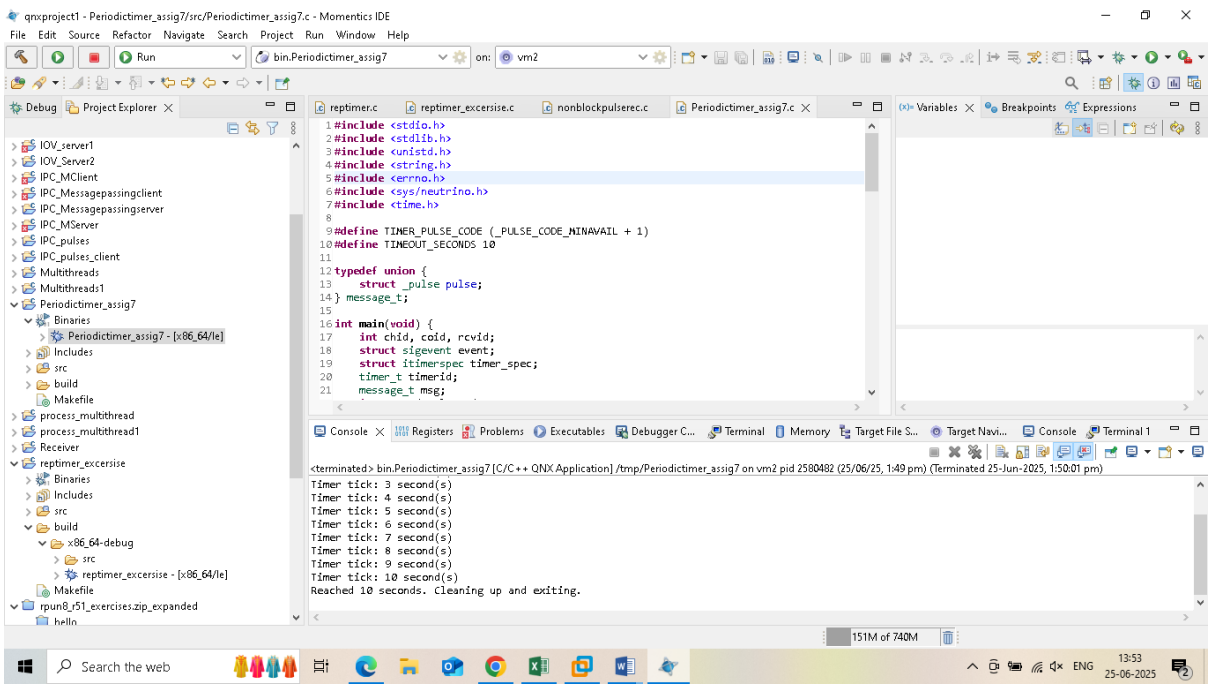
```
    ChannelDestroy(chid);
    return EXIT_FAILURE;
}

// Step 5: Set the timer - start in 1s, repeat every 1s
timer_spec.it_value.tv_sec = 1;
timer_spec.it_value.tv_nsec = 0;
timer_spec.it_interval.tv_sec = 1;
timer_spec.it_interval.tv_nsec = 0;

if (timer_settime(timerid, 0, &timer_spec, NULL) == -1) {
    perror("timer_settime");
    timer_delete(timerid);
    ConnectDetach(coid);
    ChannelDestroy(chid);
    return EXIT_FAILURE;
}

printf("Timer started. Process will terminate after %d seconds.\n", TIMEOUT_SECONDS);
```

```
// Step 6: Loop to receive timer pulses
while (1) {
    rcvid = MsgReceive(chid, &msg, sizeof(msg), NULL);
    if (rcvid == -1) {
        perror("MsgReceive");
        continue;
    }

    if (rcvid == 0 && msg.pulse.code == TIMER_PULSE_CODE) {
        seconds_elapsed++;
        printf("Timer tick: %d second(s)\n", seconds_elapsed);

        if (seconds_elapsed >= TIMEOUT_SECONDS) {
            printf("Reached %d seconds. Cleaning up and exiting.\n", TIMEOUT_SECONDS);
            break;
        }
    }
}

// Step 7: Clean up
```

```
    timer_delete(timerid);

    ConnectDetach(coid);

    ChannelDestroy(chid);



    return EXIT_SUCCESS;

}
```

Output:

```
Timer started. Process will terminate after 10 seconds.
Timer tick: 1 second(s)
Timer tick: 2 second(s)
Timer tick: 3 second(s)
Timer tick: 4 second(s)
Timer tick: 5 second(s)
Timer tick: 6 second(s)
Timer tick: 7 second(s)
Timer tick: 8 second(s)
Timer tick: 9 second(s)
Timer tick: 10 second(s)
Reached 10 seconds. Cleaning up and exiting.
```



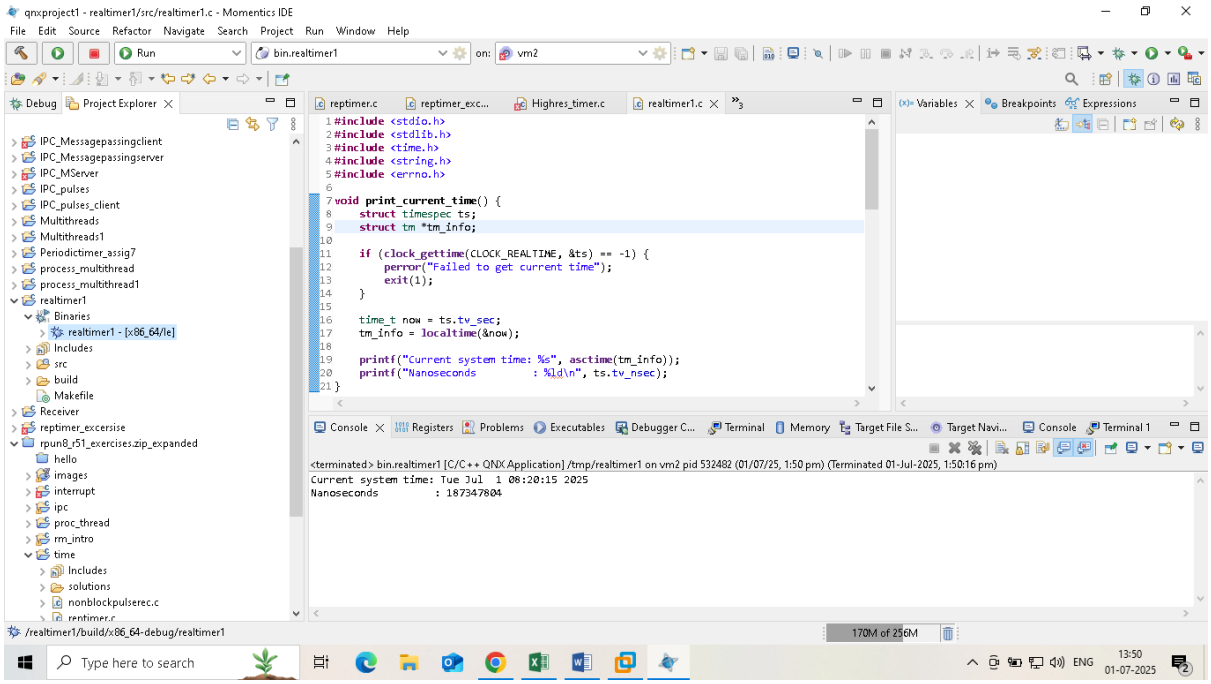
Exp:N0.11

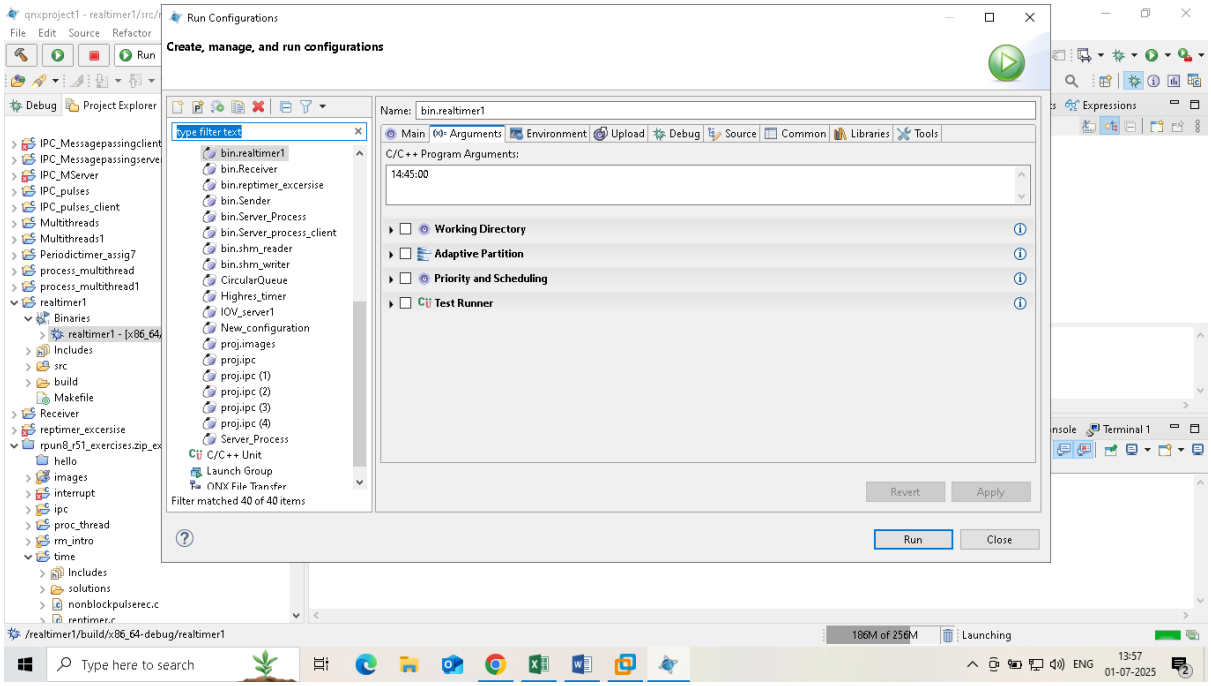
Write a program that retrieves and displays the current system time using clock_gettime and allows the user to set the system time using clock_settime by passing time value from the command prompt

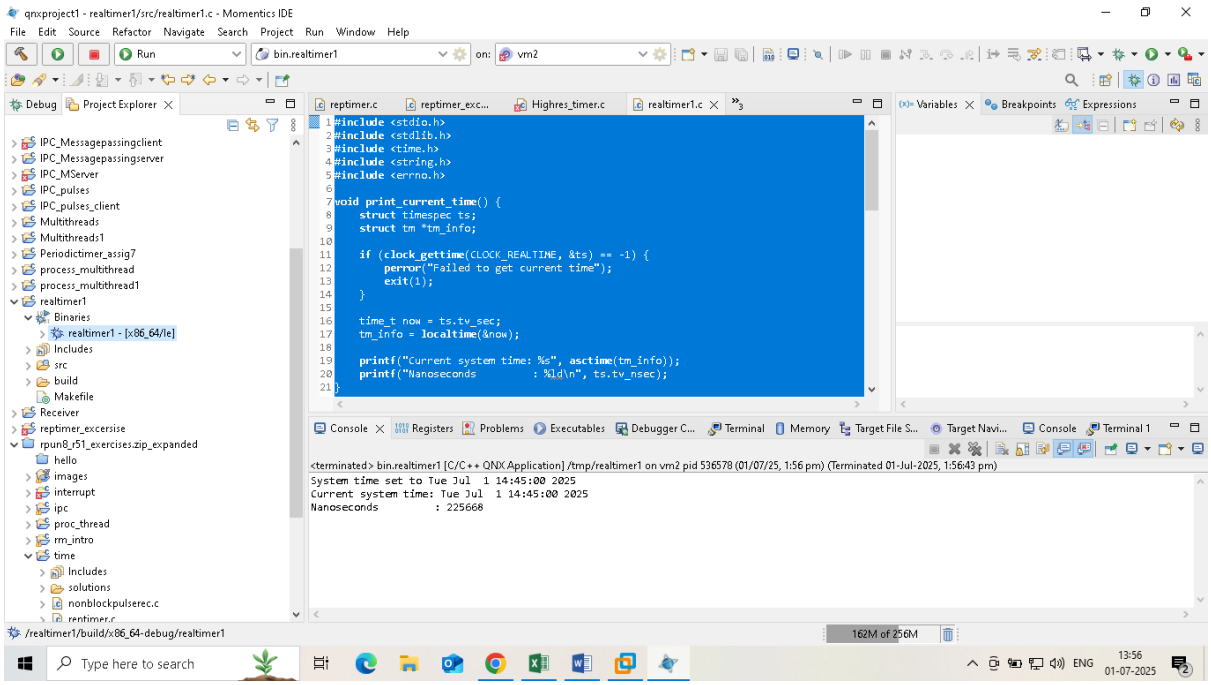
```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <errno.h>

void print_current_time() {
    struct timespec ts;
    struct tm *tm_info;

    if (clock_gettime(CLOCK_REALTIME, &ts) == -1) {
        perror("Failed to get current time");
        exit(1);
    }

    time_t now = ts.tv_sec;
    tm_info = localtime(&now);

    printf("Current system time: %s", asctime(tm_info));
    printf("Nanoseconds         : %ld\n", ts.tv_nsec);
}

int main(int argc, char *argv[]) {
    if (argc == 2) {
        int hour, minute, second;

        // Parse HH:MM:SS
        if (sscanf(argv[1], "%d:%d:%d", &hour, &minute, &second) != 3) {
            fprintf(stderr, "Invalid time format. Use HH:MM:SS\n");
            return 1;
        }

        time_t now = time(NULL);
        struct tm *timeinfo = localtime(&now);

        // Update hour, minute, second
        timeinfo->tm_hour = hour;
        timeinfo->tm_min = minute;
        timeinfo->tm_sec = second;

        // Convert back to time_t
        time_t new_time = mktime(timeinfo);
```

```
        if (new_time == -1) {
            fprintf(stderr, "Failed to convert time\n");
            return 1;
        }

        // Prepare timespec
        struct timespec ts;
        ts.tv_sec = new_time;
        ts.tv_nsec = 0;

        // Set time
        if (clock_settime(CLOCK_REALTIME, &ts) == -1) {
            perror("Failed to set system time");
            return 1;
        }

        printf("System time set to %s", asctime(timeinfo));
    }

    // Always show current time
    print_current_time();

    return 0;
}
```

Output: `Current system time: Tue Jul  1 08:20:15 2025`
`Nanoseconds         : 187347804`







Exp:N0.12

Write a Program to initialize Resource managers.

QNX Resource Manager + Timer Pulse (VMware)

🎯 What students will learn

- What an **interrupt** is
- How QNX uses **pulses** to represent interrupts
- Difference between **polling vs interrupt**
- How GPIO interrupt logic works internally

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <signal.h>

#include <time.h>


#include <sys/dispatch.h>

#include <sys/iofunc.h>

#include <sys/neutrino.h>


#define PULSE_CODE_GPIO  1


static int gpio_value = 0;


/* READ handler */

int gpio_read(resmgr_context_t *ctp, io_read_t *msg, iofunc_ocb_t
*ocb)

{

   char buffer[8];

   int len;
```

```
    len = sprintf(buffer, "%d\n", gpio_value);

    if (ocb->offset >= len)
        return 0;

    _IO_SET_READ_NBYTES(ctp, len);
    resmgr_msgwrite(ctp, buffer, len, 0);

    ocb->offset += len;
    return _RESMGR_NPARTS(0);
}

/* Pulse handler = simulated interrupt */
int pulse_handler(message_context_t *ctp, int code, unsigned flags,
void *handle)
{
    if (code == PULSE_CODE_GPIO) {
        gpio_value ^= 1;   // Toggle GPIO
        printf("INTERRUPT: GPIO toggled to %d\n", gpio_value);
    }
    return 0;
}
```

```
int main(void)
{
    dispatch_t *dpp;
    dispatch_context_t *ctp;
    resmgr_attr_t rattr;
    iofunc_attr_t ioattr;
    resmgr_connect_funcs_t connect_funcs;
    resmgr_io_funcs_t io_funcs;

    struct sigevent event;
    timer_t timerid;
    struct itimerspec timer_spec;

    /* Create dispatch */
    dpp = dispatch_create();
    if (!dpp) {
        perror("dispatch_create");
        return EXIT_FAILURE;
    }

    /* Resource manager attributes */
    memset(&rattr, 0, sizeof(rattr));
    rattr.nparts_max = 1;
    rattr.msg_max_size = 2048;
```

```
    /* File attributes */
    iofunc_attr_init(&ioattr, S_IFCHR | 0666, NULL, NULL);

    /* Init default functions */
    iofunc_func_init(_RESMGR_CONNECT_NFUNCS, &connect_funcs,
                     _RESMGR_IO_NFUNCS, &io_funcs);

    io_funcs.read = gpio_read;

    /* Attach device */
    resmgr_attach(dpp,
                  &rattr,
                  "/dev/gpio_int",
                  _FTYPE_ANY,
                  0,
                  &connect_funcs,
                  &io_funcs,
                  &ioattr);

    /* Attach pulse handler */
    dispatch_pulse_attach(dpp, -1, PULSE_CODE_GPIO, pulse_handler,
NULL);
```

```
/* Create pulse event */
event.sigev_notify = SIGEV_PULSE;
event.sigev_coid   = dispatch_coid(dpp);
event.sigev_code   = PULSE_CODE_GPIO;
event.sigev_priority = getprio(0);

timer_create(CLOCK_REALTIME, &event, &timerid);

/* Timer every 5 seconds */
timer_spec.it_value.tv_sec = 5;
timer_spec.it_value.tv_nsec = 0;
timer_spec.it_interval.tv_sec = 5;
timer_spec.it_interval.tv_nsec = 0;

timer_settime(timerid, 0, &timer_spec, NULL);

printf("GPIO Interrupt Simulation Started (/dev/gpio_int)\n");
printf("Interrupt every 5 seconds\n");

/* Dispatch loop */
ctp = dispatch_context_alloc(dpp);
while (1) {
   ctp = dispatch_block(ctp);
   dispatch_handler(ctp);
```

```
    }

    return EXIT_SUCCESS;
}
```

Output:

```
 INTERRUPT: GPIO toggled to 1
INTERRUPT: GPIO toggled to 0
```